Computer Programming Using C(PC)
DU # 2301CS101

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Unit-5

Structure & Unions, Dynamic memory allocation and Files





Prof. Vishal Kansagara
Computer Engineering Department
Darshan Institute of Engineering & Technology, Rajkot
Vishal.kansagara@darshan.ac.in
8200601076

Outline

- Structure
- Union
- File management
- Dynamic memory allocation

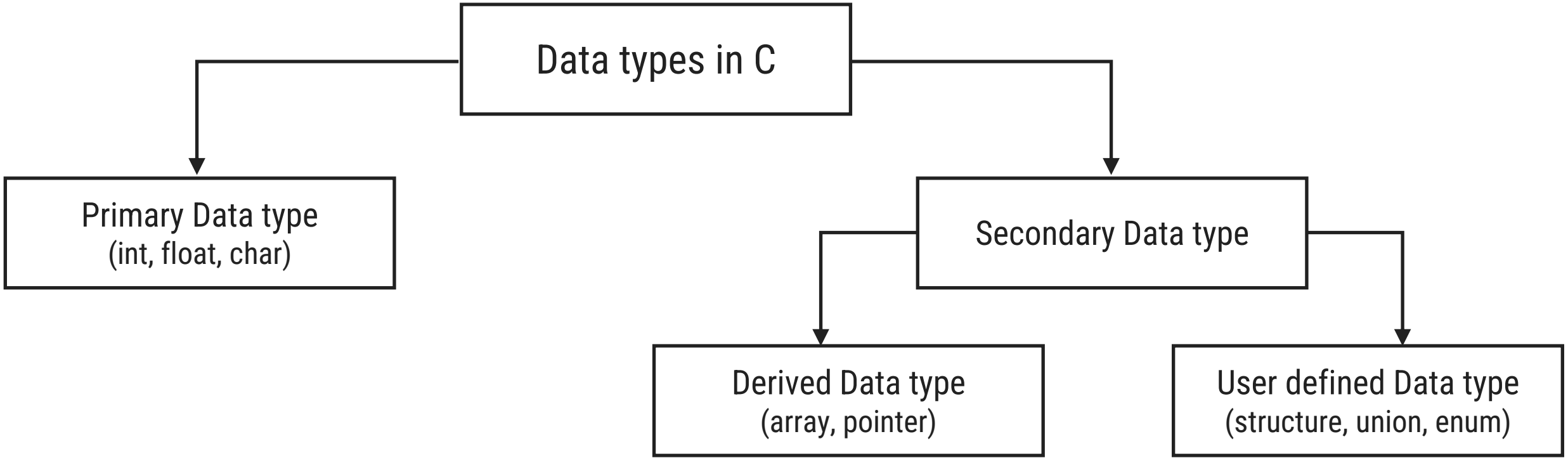
Darshan UNIVERSITY

Structure

Darshan UNIVERSITY

Data Types

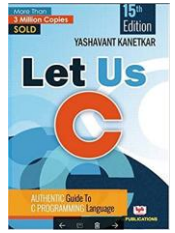
- Data types are defined as the data storage format that a variable can store a data.



- C language has built-in datatypes like primary and derived data types.
- But, still not all real world problems can be solved using those data types.
- We need custom datatype for different situation.

User Defined Datatype

- We need combination of various datatypes to understand different entity/object.
- Example-1:
  - Book



Title: Let Us C	Datatype: char / string
Author: Yashavant Kanetkar	Datatype: char / string
Page: 320	Datatype: int
Price: 255.00	Datatype: float

- Example-2:
  - Student



Name: ABC	Datatype: char / string
Roll_No: 180540107001	Datatype: int
CPI: 7.46	Datatype: float
Backlog: 01	Datatype: int

What is Structure?

- Structure is a collection of logically related data items of different datatypes grouped together under single name.
- Structure is a user defined datatype.
- Structure helps to build a complex datatype which is more meaningful than an array.
- But, an array holds similar datatype record, when structure holds different datatypes records.
- Two fundamental aspects of Structure:
  - Declaration of Structure Variable
  - Accessing of Structure Member

Darshan UNIVERSITY

Syntax to Define Structure

- To define a structure, we need to use **`struct`** keyword.
- This keyword is reserved word in C language. We can only use it for structure and its object declaration.

Syntax

```
struct structure_name
{
   member1_declaration;
   member2_declaration;
   . . .
   memberN_declaration;
};
```

structure_name is name of custom type

memberN_declaration is individual member declaration

Example to Define Structure

Example

```
struct student
{
        char name[30]; // Student Name
        int roll_no; // Student Roll No
        float CPI; // Student CPI
        int backlog; // Student Backlog
};
```

- You must terminate structure definition with semicolon ;.
- You cannot assign value to members inside the structure definition, it will cause compilation error.

Example

```
struct student
{
   char name[30] = "ABC";              // Student Name
    . . .
};
```

Create Structure variable

- A data type defines various properties about data stored in memory.
- To use any type we must declare its variable.
- Hence, let us learn how to create our custom structure type objects also known as structure variable.
- In C programming, there are two ways to declare a structure variable:
  1. Along with structure definition
  2. After structure definition.

Create Structure Variable – Cont.

- Declaration along with the structure definition

Syntax

```
struct structure_name
{
   member1_declaration;
   member2_declaration;
            . . .
   memberN_declaration;
} structure_variable;
```

Example

```
1  struct student
2  {
3      char name[30]; // Student Name
4      int roll_no; // Student Roll No
5      float CPI; // Student CPI
6      int backlog; // Student Backlog
7  } student1;
```

Darshan UNIVERSITY

Create Structure Variable – Cont.

Declaration after Structure definition

Syntax

```
struct structure_name structure_variable;
```

Example

```
1  struct student
2  {
3      char name[30]; // Student Name
4      int roll_no; // Student Roll No
5      float CPI; // Student CPI
6      int backlog; // Student Backlog
7  };
8  struct student student1; // Declare structure variable
```

Access Structure member (data)

- Structure is a complex data type, we cannot assign any value directly to it using assignment operator.
- We must assign data to individual structure members separately.
- C supports two operators to access structure members, using a structure variable.
  1. Dot/period operator (.)
  2. Arrow operator (->)

Access Structure member (data) – Cont.

1. Dot/period operator (.)

- It is known as member access operator. We use dot operator to access members of simple structure variable.

Syntax

```
structure_variable.member_name;
```

Example

```
// Assign CPI of student1
student1.CPI = 7.46;
```

2. Arrow operator (->)

- In C language it is illegal to access a structure member from a pointer to structure variable using dot operator.
- We use arrow operator to access structure member from pointer to structure.

Syntax

```
pointer_to_structure->member_name;
```

Example

```
// Student1 is a pointer to student
student1 -> CPI = 7.46;
```

Darshan UNIVERSITY

Write a program to read and display student information using structure.

Program

```c
#include <stdio.h>
struct student
{char name[40]; // Student name
int roll; // Student enrollment
float CPI; // Student mobile number
int backlog;
};
int main(){
struct student student1; // Simple structure variable
// Input data in structure members using dot operator
printf("Enter Student Name:");
scanf("%s", student1.name);
printf("Enter Student Roll Number:");
scanf("%d", &student1.roll);
printf("Enter Student CPI:");
scanf("%f", &student1.CPI);
printf("Enter Student Backlog:");
scanf("%d", &student1.backlog);
// Display data in structure members using dot operator
printf("\nStudent using simple structure variable.\n");
printf("Student name: %s\n", student1.name);
printf("Student Enrollment: %d\n", student1.roll);
printf("Student CPI: %f\n", student1.CPI);
printf("Student Backlog: %i\n", student1.backlog);
}
```

Output

```
Enter Student Name:aaa
Enter Student Roll Number:111
Enter Student CPI:7.89
Enter Student Backlog:0

Student using simple structure variable.
Student name: aaa
Student Enrollment: 111
Student CPI: 7.890000
Student Backlog: 0
```

Darshan UNIVERSITY

WAP to declare time structure and read two different time period and display sum of it.

```c
#include<stdio.h>
struct time {
int hours;
int minutes;
int seconds;
};
int main() {
struct time t1,t2;
int h, m, s;
//1st time
printf ("Enter 1st time.");
printf ("\nEnter Hours: ");
scanf ("%d",&t1.hours);
printf ("Enter Minutes: ");
scanf ("%d",&t1.minutes);
printf ("Enter Seconds: ");
scanf ("%d",&t1.seconds);
printf ("The Time is
%d:%d:%d",t1.hours,t1.minutes,t1.seconds);
//2nd time
printf ("\n\nEnter the 2nd time.");
printf ("\nEnter Hours: ");
scanf ("%d",&t2.hours);
printf ("Enter Minutes: ");
scanf ("%d",&t2.minutes);
printf ("Enter Seconds: ");
scanf ("%d",&t2.seconds);
printf ("The Time is
%d:%d:%d",t2.hours,t2.minutes,t2.seconds);
h = t1.hours + t2.hours;
m = t1.minutes + t2.minutes;
s = t1.seconds + t2.seconds;
printf ("\nSum of the two time's is
%d:%d:%d",h,m,s);
return 0;}
```

Output

```
Enter 1st time.
Enter Hours: 1
Enter Minutes: 20
Enter Seconds: 20
The Time is 1:20:20

Enter the 2nd time.
Enter Hours: 2
Enter Minutes: 10
Enter Seconds: 10
The Time is 2:10:10
Sum of the two time's is 3:30:30
```

Darshan UNIVERSITY

Array of Structure

- It can be defined as the collection of multiple structure variables where each variable contains information about different entities.
- The array of structures in C are used to store information about multiple entities of different data types.

Syntax

```
struct structure_name
{
        member1_declaration;
        member2_declaration;
        ...
        memberN_declaration;
} structure_variable[size];
```

Darshan UNIVERSITY

WAP to read and display N student information using array of structure.

```c
#include<stdio.h>
struct student {
char name[20];
int rollno;
float cpi;
};
int main(){
int i,n;
printf("Enter how many records u want to store : ");
scanf("%d",&n);
struct student sarr[n];
for(i=0; i<n; i++){
printf("\nEnter %d record : \n",i+1);
printf("Enter Name : ");
scanf("%s",sarr[i].name);
printf("Enter RollNo. : ");
scanf("%d",&sarr[i].rollno);
printf("Enter CPI : ");
scanf("%f",&sarr[i].cpi);
}
printf("\n\tName\tRollNo\tMarks\t\n");
for(i=0; i<n; i++) {
printf("\t%s\t\t%d\t\t%.2f\t\n", sarr[i].name,
sarr[i].rollno, sarr[i].cpi);
}
return 0;}
```

```
Enter how many records u want to store : 3

Enter 1 record :
Enter Name : aaa
Enter RollNo. : 111
Enter CPI : 7.89

Enter 2 record :
Enter Name : bbb
Enter RollNo. : 222
Enter CPI : 7.85

Enter 3 record :
Enter Name : ccc
Enter RollNo. : 333
Enter CPI : 8.56

   Name    RollNo  Marks
   aaa     111     7.89
   bbb     222     7.85
   ccc     333     8.56
```

WAP to declare time structure and read two different time period and display it using function.

```c
#include<stdio.h>
struct Time {
int hours;
int minutes;
int seconds;
};
struct Time input(); // function declaration
int main()
{
struct Time t;
t=input();
printf("Hours : Minutes : Seconds\n %d : %d :
%d",t.hours,t.minutes,t.seconds);
return 0;
}
struct Time input() // function definition
{
struct Time tt;
printf ("Enter Hours: ");
scanf ("%d",&tt.hours);
printf ("Enter Minutes: ");
scanf ("%d",&tt.minutes);
printf ("Enter Seconds: ");
scanf ("%d",&tt.seconds);
return tt; // return structure variable
}
```

Output

```
Enter Hours: 1
Enter Minutes: 20
Enter Seconds: 20
Hours : Minutes : Seconds
1 : 20 : 20
```

Structure using Pointer

- Reference/address of structure object is passed as function argument to the definition of function.

Program

```c
#include <stdio.h>
struct student {
char name[20];
int rollno;
float cpi;
};
int main()
{
struct student *studPtr, stud1;
studPtr = &stud1;
printf("Enter Name: ");
scanf("%s", studPtr->name);
printf("Enter RollNo: ");
scanf("%d", &studPtr->rollno);
printf("Enter CPI: ");
scanf("%f", &studPtr->cpi);
printf("\nStudent Details:\n");
printf("Name: %s\n", studPtr->name);
printf("RollNo: %d", studPtr->rollno);
printf("\nCPI: %f", studPtr->cpi);
return 0;
}
```

Output

```
Enter Name: ABC
Enter RollNo: 121
Enter CPI: 7.46

Student Details:
Name: ABC
RollNo: 121
CPI: 7.460000
```

Nested Structure

- When a structure contains another structure, it is called nested structure.
- For example, we have two structures named Address and Student. To make Address nested to Student, we have to define Address structure before and outside Student structure and create an object of Address structure inside Student structure.

Syntax

```
struct structure_name1
{
    member1_declaration;
    member2_declaration;
    ...
    memberN_declaration;
};
struct structure_name2
{
    member1_declaration;
    member2_declaration;
    ...
    struct structure1 obj;
};
```

WAP to read and display student information using nested of structure.

```c
#include<stdio.h>
struct Address
{
char HouseNo[25];
char City[25];
char PinCode[25];
};
struct Student
{
char name[25];
int roll;
float cpi;
struct Address Add;
};
int main()
{
int i;
struct Student s;
printf("\n\tEnter Student Name : ");
scanf("%s",s.name);
printf("\n\tEnter Student Roll Number : ");
scanf("%d",&s.roll);
printf("\n\tEnter Student CPI : ");
scanf("%f",&s.cpi);
printf("\n\tEnter Student House No : ");
scanf("%s",s.Add.HouseNo);
printf("\n\tEnter Student City : ");
scanf("%s",s.Add.City);
printf("\n\tEnter Student Pincode : ");
scanf("%s",s.Add.PinCode);
printf("\nDetails of Students");
printf("\n\tStudent Name : %s",s.name);
printf("\n\tStudent Roll Number : %d",s.roll);
printf("\n\tStudent CPI : %f",s.cpi);
printf("\n\tStudent House No : %s",s.Add.HouseNo);
printf("\n\tStudent City : %s",s.Add.City);
printf("\n\tStudent Pincode : %s",s.Add.PinCode);
return 0;
}
```

Output

```
Details of Students
        Student Name : aaa
        Student Roll Number : 111
        Student CPI : 7.890000
        Student House No : 39
        Student City : rajkot
        Student Pincode : 360001
```

Practice Programs

1. Define a structure data type called time_struct containing three member's integer hours, minutes, second. Develop a program that would assign values to individual member and display the time in following format : HH:MM:SS
2. WAP to create structure of book with book title, author name, publication, and price. Read data of n books and display them.
3. Define a structure Person that would contain person name, date of joining, and salary using this structure to read this information of 5 people and print the same on screen.
4. Define a structure time_struct containing three member's integer hour, integer minute and integer second. WAP that would assign values to the individual number and display the time in the following format: 16: 40: 51.
5. Define a structure cricket that will describe the following information:

   Player name

   Team name

   Batting average
6. Using cricket, declare an array player with 50 elements and WAP to read the information about all the 50 players and print team wise list containing names of players with their batting average.
7. Define a structure student_record to contain name, branch, and total marks obtained. WAP to read data for 10 students in a class and print them.

Union

What is Union?

- Union is a user defined data type similar like Structure.
- It holds different data types in the same memory location.
- You can define a union with various members, but only one member can hold a value at any given time.
- Union provide an efficient way of using the same memory location for multiple-purpose.

Syntax to Define and Access Union

- Declaration of union must start with the keyword `union` followed by the union name and union's member variables are declared within braces.

Syntax

```
union union_name
{
    member1_declaration;
    member2_declaration;
    . . .
    memberN_declaration;
};
```

union_name is name of custom type.

memberN_declaration is individual member declaration.

- Accessing the union members:
  - You need to create an object of union to access its members.
  - Object is a variable of type union. Union members are accessed using the dot operator(.) between union's object and union's member name.

Syntax

```
union union_name union_variable;
```

Example to Define Union

Example

```
1  union student
2  {
3      char name[30]; // Student Name
4      int roll_no; // Student Roll No
5      float CPI; // Student CPI
6      int backlog; // Student Backlog
7  } student1;
```

- You must terminate union definition with semicolon ;.
- You cannot assign value to members inside the union definition, it will cause compilation error.

Example

```
1  union student
2  {
3      char name[30] = “ABC”; // Student Name
4      . . .
5  } student1;
```

Structure Vs. Union

COMPARISON	STRUCTURE	UNION
Basic	The separate memory location is allotted to each member of the structure.	All members of the 'union' share the same memory location.
keyword	'struct'	'union'
Size	Size of Structure = sum of size of all the data members.	Size of Union = size of the largest member.
Store Value	Stores distinct values for all the members.	Stores same value for all the members.
At a Time	A structure stores multiple values, of the different members, of the structure.	A union stores a single value at a time for all members.

Declaration	`struct ss`
	`{`
	`  int a;`
	`  float f;`
	`  char c`
	`};`
		`union uu`
		`{`
		`  int a;`
		`  float f;`
		`  char c`
		`};`

Where Union should be used?

- Mouse Programming
- Embedded Programming
- Low Level System Programming

File management

Darshan UNIVERSITY

File management is what you have, and how you want to manipulate it. - Anonymous

Why File Management?

- In real life, we want to store data permanently so that later we can retrieve it and reuse it.
- A file is a collection of characters stored on a secondary storage device like hard disk, or pen drive.
- There are two kinds of files that programmer deals with:
  - Text Files are human readable and it is a stream of plain English characters
  - Binary Files are computer readable, and it is a stream of processed characters and ASCII symbols

Text File

```
Hello, this is a text file. Whatever written here can be read easily without the help of a computer.
```

Binary File

```
11010011010100010110111010101
11010111010011010100010110111
01010111010111010011
```

File Opening Modes

- We can perform different operations on a file based on the file opening modes

Mode	Description
r	Open the file for reading only. If it exists, then the file is opened with the current contents; otherwise an error occurs.
w	Open the file for writing only. A file with specified name is created if the file does not exists. The contents are deleted, if the file already exists.
a	Open the file for appending (or adding data at the end of file) data to it. The file is opened with the current contents safe. A file with the specified name is created if the file does not exists.
r+	The existing file is opened to the beginning for both reading and writing.
w+	Same as w except both for reading and writing.
a+	Same as a except both for reading and writing.

Note: The main difference is w+ truncate the file to zero length if it exists or create a new file if it doesn't. While r+ neither deletes the content nor create a new file if it doesn't exist.

File Handling Functions

- Basic file operation performed on a file are opening, reading, writing, and closing a file.

Syntax	Description
fp=fopen(file_name, mode);	This statement opens the file and assigns an identifier to the FILE type pointer fp. Example: fp = fopen("printfile.c","r");
fclose(filepointer);	Closes a file and release the pointer. Example: fclose(fp);
fprintf(fp, “control string”, list);	Here fp is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: fprintf(fp, "%s %d %c", name, age, gender);

File Handling Functions

Syntax	Description
fscanf(fp, “control string”, list);	Here fp is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: fscanf(fp, "%s %d", &item, &qty);
int getc(FILE *fp);	getc() returns the next character from a file referred by fp; it require the FILE pointer to tell from which file. It returns EOF for end of file or error. Example: c = getc(fp);
int putc(int c, FILE *fp);	putc() writes or appends the character c to the FILE fp. If a putc function is successful, it returns the character written, EOF if an error occurs. Example: putc(c, fp);

File Handling Functions

Syntax	Description
int getw(FILE *pvar);	getw() reads an integer value from FILE pointer fp and returns an integer. Example: i = getw(fp);
putw(int, FILE *fp);	putw writes an integer value read from terminal and are written to the FILE using fp. Example: putw(i, fp);
EOF	EOF stands for “End of File”. EOF is an integer defined in <stdio.h> Example: while(ch != EOF)

Write a C program to display content of a given file.

Program

```c
#include <stdio.h>
void main()
{
    FILE *fp; //p is a FILE type pointer
    char ch; //ch is used to store single character
    fp = fopen("file1.c","r");//open file in read mode and store file pointer in p
    do {  //repeat step 9 and 10 until EOF is reached
        ch = getc(fp); //get character pointed by p into ch
        putchar(ch); //print ch value on monitor
    }while(ch != EOF); //condition to check EOF is reached or not
    fclose(fp); //free up the file pointer pointed by fp
}
```

Darshan UNIVERSITY

Write a C program to copy a given file.

Program

```c
#include <stdio.h>
void main()
{
    FILE *fp1, *fp2; //p and q is a FILE type pointer
    char ch; //ch is used to store temporary data
    fp1 = fopen("file1.c","r"); //open file “file1.c” in read mode
    fp2 = fopen("file2.c","w"); //open file “file2.c” in write mode
    do { //repeat step 9 and 10 until EOF is reached
        ch = getc(fp1); //get character pointed by p into ch
        putc(ch, fp2); //print ch value into file, pointed by pointer q
    }while(ch != EOF); //condition to check EOF is reached or not
    fclose(fp1); //free up the file pointer p
    fclose(fp2); //free up the file pointer q
    printf("File copied successfully...");
}
```

Darshan UNIVERSITY

File Positioning Functions

- fseek, ftell, and rewind functions will set the file pointer to new location.
- A subsequent read or write will access data from the new position.

Syntax	Description
fseek(FILE *fp, long offset, int position);	fseek() function is used to move the file position to a desired location within the file. fp is a FILE pointer, offset is a value of datatype long, and position is an integer number. Example: /* Go to the end of the file, past the last character of the file */ fseek(fp, 0, 2);
long ftell(FILE *fp);	ftell takes a file pointer and returns a number of datatype long, that corresponds to the current position. This function is useful in saving the current position of a file. Example: /* n would give the relative offset of the current position. */ n = ftell(fp);

File Positioning Functions

Syntax	Description
rewind(fp);	rewind() takes a file pointer and resets the position to the start of the file. Example: /* The statement would assign 0 to n because the file position has been set to the start of the file by rewind. */ rewind(fp);

Write a C program to count lines, words, tabs, and characters

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      FILE *p;
5      char ch;
6      int ln=0,t=0,w=0,c=0;
7      p = fopen("text1.txt","r");
8      ch = getc(p);
9      while (ch != EOF) {
10         if (ch == '\n')
11             ln++;
12         else if(ch == '\t')
13             t++;
14         else if(ch == ' ')
15             w++;
16         else
```

Program (contd.)

```
17                 c++;
18
19             ch = getc(p);
20     }
21     fclose(p);
22     printf("Lines = %d, tabs = %d, w
   ords = %d, characters = %d\n",ln, t,
    w, c);
23 }
```

Output

```
Lines = 22, tabs = 0, words = 152, characters = 283
```

Darshan UNIVERSITY

Practice Programs

1) Write a C program to write a string in file.
2) A file named data contains series of integer numbers. Write a C program to read all numbers from file and then write all the odd numbers into file named “odd” and write all even numbers into file named “even”. Display all the contents of these file on screen.
3) Write a C program to read name and marks of n number of students and store them in a file.
4) Write a C program to print contents in reverse order of a file.
5) Write a C program to compare contents of two files.
6) Write a C program to copy number of bytes from a specific offset to another file.
7) Write a C program to convert all characters in UPPER CASE of a File.

Dynamic memory allocation

Darshan UNIVERSITY

Dynamic Memory Allocation (DMA)

- If memory is allocated at runtime (during execution of program) then it is called dynamic memory.
- It allocates memory from heap (*heap*: it is an empty area in memory).
- Memory can be accessed only through a pointer.

When DMA is needed?

- It is used when number of variables are not known in advance or large in size.
- Memory can be allocated at any time and can be released at any time during runtime.

malloc() function

- `malloc ()` is used to allocate a fixed amount of memory during the execution of a program.
- `malloc ()` allocates `size_in_bytes` of memory from heap, if the allocation succeeds, a pointer to the block of memory is returned else `NULL` is returned.
- Allocated memory space may not be contiguous.
- Each block contains a `size`, a pointer to the next block, and the space itself.
- The blocks are kept in ascending order of storage address.
- The memory is not initialized.

Syntax	Description
`ptr_var = (cast_type *) malloc (size_in_bytes);`	This statement returns a pointer to `size_in_bytes` of uninitialized storage, or `NULL` if the request cannot be satisfied. Example: `fp = (int *)malloc(sizeof(int) *20);`

Write a C program to allocate memory using malloc.

Program

```c
#include <stdio.h>
void main()
{
    int *fp; //fp is a pointer variable
    fp = (int *)malloc(sizeof(int)); //returns a pointer to int size storage
    *fp = 25; //store 25 in the address pointed by fp
    printf("%d", *fp); //print the value of fp, i.e. 25
    free(fp); //free up the space pointed to by fp
}
```

C

Output

```
25
```

calloc() function

- **`calloc()`** is used to allocate a block of memory during the execution of a program.
- **`calloc()`** allocates a region of memory to hold `no_of_blocks` of `size_of_block` each, if the allocation succeeds then a pointer to the block of memory is returned else `NULL` is returned.
- The memory is initialized to ZERO.

Syntax	Description

`ptr_var = (cast_type *) calloc (no_of_blocks, size_of_block);`	This statement returns a pointer to `no_of_blocks` of size `size_of_blocks`, it returns `NULL` if the request cannot be satisfied.
	
	Example:
	`int n = 20;`
	`fp = (int *)calloc(n, sizeof(int));`

Write a C program to allocate memory using calloc.

Program

```c
#include <stdio.h>
void main()
{
    int i, n; //i, n are integer variables
    int *fp; //fp is a pointer variable
    printf("Enter how many numbers: ");
    scanf("%d", &n);
    fp = (int *)calloc(n, sizeof(int)); //calloc returns a pointer to n blocks
    for(i = 0; i < n; i++)  //loop through until all the blocks are read
    {
        scanf("%d",fp); //read and store into location where fp points
        fp++; //increment the pointer variable
    }
    free(fp); //frees the space pointed to by fp
}
```

realloc() function

- `realloc()` changes the size of the object pointed to by pointer fp to specified size.
- The contents will be unchanged up to the minimum of the old and new sizes.
- If the new size is larger, the new space will be uninitialized.
- `realloc()` returns a pointer to the new space, or `NULL` if the request cannot be satisfied, in which case `*fp` is unchanged.

Syntax	Description
`ptr_var = (cast_type *) realloc (void *fp, size_t);`	This statement returns a pointer to new space, or NULL if the request cannot be satisfied. Example: `fp = (int *)realloc(fp,sizeof(int)*20);`

Write a C program to allocate memory using realloc.

Program

```c
#include <stdio.h>
void main()
{
    int *fp; //fp is a file pointer
    fp = (int *)malloc(sizeof(int)); //malloc returns a pointer to int size storage
    *fp = 25; //store 25 in the address pointed by fp
    fp =(int *)realloc(fp, 2*sizeof(int)); //returns a pointer to new space
    printf("%d", *fp); //print the value of fp
    free(fp); //free up the space pointed to by fp
}
```

Output

```
25
```

free() function

- `free` deallocates the space pointed to by fp.
- It does nothing if fp is `NULL`.
- fp must be a pointer to space previously allocated by `calloc`, `malloc` or `realloc`.

Syntax	Description
`void free(void *);`	This statement free up the memory not needed anymore. Example: `free(fp);`

Write a C program to sort numbers using malloc

Program

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int i,j,t,n;
    int *p;
    printf("Enter value of n: ");
    scanf("%d", &n);
    p=(int *) malloc(n * sizeof(int));
    printf("Enter values\n");
    for(i=0; i<n; i++)
        scanf("%d", &p[i]);
    for(i=0; i<n; i++)
    {
        for(j= i+1; j<n; j++)
        {
```

Program (cont.)

```c
            if(p[i] > p[j])
            {
                t = p[i];
                p[i] = p[j];
                p[j] = t;
            }
        }
    }
    printf("Ascending order\n");
    for(i=0; i<n; i++)
        printf("%d\n", p[i]);
    free(p);
}
```

Darshan UNIVERSITY

Write a C program to find square of numbers using calloc

Program

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int i,n;
    int *p;
    printf("Enter value of n: ");
    scanf("%d",&n);
    p=(int*)calloc(n,sizeof(int));
    printf("Enter values\n");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    for(i=0;i<n;i++)
        printf("Square of %d = %d\n", p[i], p[i] * p[i]);
    free(p);
}
```

Output

```
Enter value of n: 3
Enter values
3
2
5
Square of 3 = 9
Square of 2 = 4
Square of 5 = 25
```

Darshan UNIVERSITY

Write a C program to add/remove item from a list using realloc

Program

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int i, n1, n2;
    int *fp;
    printf("Enter size of list: ");
    scanf("%d", &n1);
    fp=(int *) malloc (n1 * sizeof(int));

    printf("Enter %d numbers\n", n1);
    for(i = 0; i < n1; i++)
        scanf("%d", &fp[i]);

    printf("The numbers in the list are\n");
    for(i = 0; i < n1; i++)
        printf("%d\n", fp[i]);
```

Program (cont.)

```c
    printf("Enter new size of list: ");
    scanf("%d", &n2);

    fp = realloc(fp, n2 * sizeof(int));
    if(n2 > n1)
    {
        printf("Enter %d numbers\n", n2 - n1);
        for(i = n1; i < n2; i++)
            scanf("%d", &fp[i]);
    }
    printf("The numbers in the list are\n");
    for(i = 0; i < n2; i++)
        printf("%d\n", fp[i]);
}
```

Darshan UNIVERSITY

Practice Programs

1) Write a C program to calculate sum of n numbers entered by user.
2) Write a C program to input and print text using DMA
3) Write a C program to read and print student details using structure and DMA

Thank you

Darshan UNIVERSITY